INFORME EJECUTIVO DEL PROYECTO

USTA - Proyecto Integrador de Teoría del Riesgo Python para Python para desarrollo de Apis e IA

Autoras	Laura Alejandra Sepúlveda e Ingrid Johana Umbacía Ramírez
Asignatura	Teoría del Riesgo y Python para desarrllo de apis e IA
Programa	Universidad Santo Tomás - Facultad de Estadística
Base documental	Código fuente del backend y frontend, README, guion de sustentación, HTML de requerimientos y videos de demostración
Fecha	18 de abril de 2026

Propósito del documento. Este informe presenta una valoración ejecutiva del proyecto con base en la rúbrica consignada en el archivo HTML y en los materiales entregados. La redacción se limita a información verificable; por esa razón, cuando una cifra solo puede confirmarse en la ejecución mostrada, se incorpora como resultado observado en la corrida revisada.

1. Síntesis ejecutiva

El proyecto corresponde a un tablero de análisis de riesgo financiero construido con una arquitectura de dos capas: un backend en FastAPI y un frontend en Streamlit. La evidencia revisada muestra que el backend concentra la descarga de datos, la validación de entradas, el cálculo de métricas y la publicación de resultados mediante endpoints, mientras que el frontend organiza la navegación del usuario por módulos temáticos.

A la luz del código y de la ejecución revisada, el proyecto sí cubre los componentes principales solicitados: análisis técnico, rendimientos, ARCH/GARCH, CAPM, VaR y CVaR, optimización de Markowitz, señales automáticas, contexto macroeconómico y una capa API documentada. La fortaleza central no radica solo en la variedad de métricas, sino en haberlas articulado en una estructura reproducible y relativamente bien desacoplada.

2. Alcance y coherencia con la rúbrica

El HTML de requerimientos establece, entre otros puntos, la necesidad de un backend con FastAPI, modelos Pydantic para requests y responses, inyección de dependencias con Depends(), configuración con BaseSettings, un tablero por módulos y un informe ejecutivo breve. La evidencia entregada permite confirmar el cumplimiento sustantivo de esos componentes. En main.py aparecen los endpoints raíz, activos, precios, rendimientos, indicadores, var, capm, frontera-eficiente, alertas y macro; esa cobertura supera el mínimo de siete endpoints exigido. En models.py se observan modelos tipados para entradas y salidas, además de validadores para tickers, períodos, pesos y condiciones de consistencia del portafolio. En config.py y dependencies.py se aprecia el uso de BaseSettings, lru_cache y Depends() para desacoplar configuración y servicios.

Desde la perspectiva de la rúbrica, el proyecto no parece limitado a una maqueta visual. Existe una implementación real de servicios, validación y exposición de resultados, lo que fortalece los criterios de Backend FastAPI y Pydantic, calidad del tablero y buenas prácticas de desarrollo.

3. Arquitectura técnica verificada

La arquitectura observada separa con claridad presentación y lógica de negocio. El frontend consume información del backend mediante llamadas HTTP y maneja errores de conexión cuando la API no está disponible. En services.py se identifican clases diferenciadas para acceso a datos, cálculo de riesgo, indicadores técnicos y análisis de portafolio. También se evidencian dos decoradores personalizados: uno para registrar tiempos de ejecución y otro para cachear resultados con TTL. Esa decisión es pertinente en un proyecto que consulta fuentes externas de mercado, porque reduce llamadas repetidas y mejora trazabilidad.

En VaRRequest se verifica una validación cruzada especialmente importante: la cantidad de tickers debe coincidir con la cantidad de pesos y los pesos deben sumar aproximadamente 1.0. Desde el punto de vista metodológico, este control es básico pero indispensable, pues evita ejecutar métricas de riesgo sobre portafolios mal definidos. La estructura general del repositorio, además, sigue la organización sugerida en la rúbrica: archivos separados para configuración, dependencias, modelos, servicios y aplicación principal.

4. Desarrollo analítico por módulos

La ejecución revisada confirma que el tablero despliega los módulos previstos en la navegación lateral: portada y contexto, análisis técnico, rendimientos, ARCH/GARCH, CAPM y beta, VaR y CVaR, Markowitz, señales, macro y benchmark, y la sección de API. En el módulo de análisis técnico se observan indicadores como SMA, EMA, RSI, MACD, Bandas de Bollinger y estocástico, coherentes con lo solicitado. El módulo de rendimientos muestra la caracterización estadística y las pruebas de normalidad. En ARCH/GARCH se comparan ARCH(1), GARCH(1,1), EGARCH y GJR-GARCH, y en la corrida revisada GARCH(1,1) aparece como mejor modelo tanto por AIC como por BIC.

En CAPM se observa una tabla para todos los activos, con beta, alpha diario, R cuadrado y descomposición del riesgo. El contenido mostrado deja ver un patrón claro: TSM aparece como el activo más agresivo de la muestra, con beta superior a uno, mientras que JNJ, PG y UL se comportan como activos defensivos con betas muy bajas. Este resultado es consistente con la composición sectorial del portafolio y con la lógica del modelo. El módulo de VaR y CVaR, por su parte, presenta tres enfoques: paramétrico, histórico y Montecarlo, además del test de Kupiec para backtesting. En la corrida revisada, el VaR histórico diario al 95 % resulta menor que el paramétrico y el Montecarlo, y el test de Kupiec clasifica el ajuste como válido.

El módulo de benchmark muestra una comparación frente al S&P 500 con métricas adicionales. El panel revisado permite observar que el portafolio exhibe mejor rendimiento anualizado y menor volatilidad anual que el índice de referencia en la corrida mostrada, junto con un alpha de Jensen positivo y un máximo drawdown menos severo. En consecuencia, la comparación relativa no solo está implementada, sino que aporta una lectura de desempeño ajustado por riesgo.

5. Hallazgos observados en la corrida mostrada

Los materiales revisados permiten registrar varios resultados concretos. En CAPM, la tasa libre de riesgo visible es 3.60 % anual y la prima de mercado reportada es 14.15 % anual frente al S&P 500. En la misma corrida, TSM presenta beta aproximada de 1.6744, mientras que MSI se ubica alrededor de 0.5123 y XOM cerca de 0.3246; JNJ, PG y UL muestran betas aún menores. Esto sugiere una estructura donde solo uno de los activos amplifica claramente los movimientos del mercado, mientras el resto actúa como componente más defensivo o de baja sensibilidad.

En VaR, la corrida observada registra, para el portafolio mostrado y al 95 %, un VaR diario paramétrico cercano a 1.2285 %, un VaR histórico de 1.0618 % y un VaR Montecarlo de 1.2365 %. El backtesting asociado muestra 10 violaciones observadas frente a 7.5 esperadas, una tasa de violaciones de 6.667 %, un estadístico LR de Kupiec de 0.797760 y una lectura de modelo válido. En benchmark, las cifras visibles señalan un rendimiento anualizado del

portafolio de 20.37 % frente a 17.84 % del S&P 500, volatilidad anual de 12.37 % frente a 16.30 %, Sharpe de 1.3559 frente a 0.8735, máximo drawdown de -14.76 % frente a -19.36 %, alpha de Jensen anual de 11.3016 %, beta de 0.4514, tracking error de 13.37 % e information ratio de 0.1171.

Estas cifras deben leerse con prudencia: corresponden a una corrida concreta revisada y no necesariamente a un resultado fijo e inmutable, porque el proyecto descarga datos de mercado dinámicos. Sin embargo, muestran que el sistema no solo calcula, sino que también produce salidas cuantitativas interpretables.

6. Valoración crítica

El proyecto presenta varias fortalezas. En primer lugar, integra teoría del riesgo con prácticas actuales de desarrollo para APIs, algo que no siempre se consigue en trabajos académicos de este nivel. En segundo lugar, la modularidad del código facilita la lectura y la defensa técnica del sistema. En tercer lugar, la existencia de backtesting con Kupiec, validaciones Pydantic y manejo explícito de errores muestra un cuidado superior al de una simple visualización.

También existen límites. Parte del frontend todavía realiza cálculos o consultas auxiliares localmente, lo que puede restar pureza al criterio de que toda la lógica pase por el backend. Además, aunque los materiales revisados permiten recuperar varias cifras, el cierre documental original del proyecto no consolidaba todos los resultados empíricos en un solo informe. Finalmente, el uso de Yahoo Finance resulta razonable para un ejercicio académico, pero deja abierta la posibilidad de fortalecer la capa macroeconómica con fuentes oficiales como FRED o Banco de la República, tal como sugería el HTML de requerimientos.

7. Conclusión

Con base en los materiales revisados, el proyecto RiskLab USTA cumple de manera sólida con el espíritu y con buena parte de la letra de la rúbrica. No se trata únicamente de un tablero vistoso: existe una arquitectura funcional en FastAPI, validación formal de entradas, servicios diferenciados para los cálculos y una interfaz que organiza de forma pedagógica los conceptos del curso. La corrida revisada permite además documentar resultados concretos en CAPM, VaR/Kupiec y benchmark, lo que fortalece la trazabilidad del análisis.

En síntesis, el trabajo evidencia comprensión metodológica y capacidad de implementación. Su principal valor está en haber convertido conceptos como beta, VaR, CVaR, volatilidad condicional y frontera eficiente en un sistema reproducible, defendible y conectado a datos de mercado. Para una entrega final todavía más robusta, convendría conservar una corrida de cierre con resultados exportados y fechados, de modo que la sustentación y el informe usen exactamente la misma base cuantitativa.

8. Cuadro sintético de cumplimiento

Componente	Evidencia verificada	Valoración
Backend FastAPI	Endpoints, Depends(), BaseSettings, /docs, modelos request/response	Cumplimiento alto
Pydantic y validación	Field(), field_validator, model_validator en entradas sensibles	Cumplimiento alto
Módulos del tablero	Navegación por análisis técnico, rendimientos, GARCH, CAPM, VaR, Markowitz, señales, benchmark	Cumplimiento alto
Resultados cuantitativos	Cifras observables en videos para CAPM, VaR/Kupiec y benchmark	Cumplimiento medio-alto
Buenas prácticas	requirements, README,	Cumplimiento alto

	separación backend/frontend, caché y logging	

9. Ilustraciones seleccionadas

Se incluyen capturas puntuales porque aportan soporte directo a los resultados cuantitativos descritos en el informe. No se agregan más imágenes para evitar que el documento pierda su carácter ejecutivo.

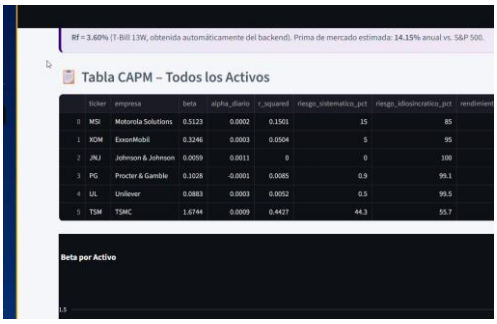

Figura 1. Módulo CAPM con tabla de betas observada en video.

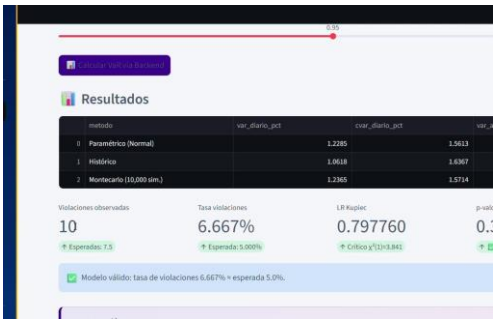

Figura 2. Módulo VaR y CVaR con resultados y test de Kupiec.

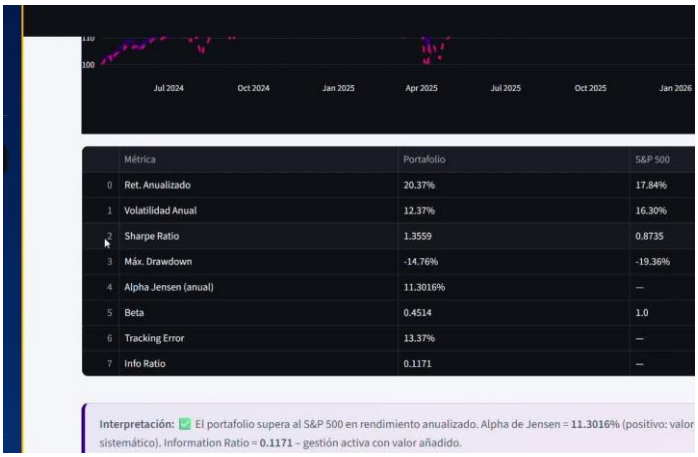

Figura 3. Módulo benchmark con métricas de desempeño relativo.